

Estilos Arquitectónicos

Catálogo POSA: *dataflow* · *distributed* · *interactive* · *event-based*

Ingeniería de Software II · ITBA · 2026-1C · Repaso conceptual

RESUMEN

Un **estilo arquitectónico** es una solución reutilizable a un problema recurrente de organización de componentes, con trade-offs conocidos. Elegir un estilo es una decisión arquitectónica significativa. Catálogo POSA en 4 familias: *dataflow*, *distributed*, *interactive* y *event-based*.

DEFINICIÓN

Estilo (o pattern) arquitectónico: solución reutilizable a un problema recurrente de organización de componentes, con **trade-offs conocidos**. Es el vocabulario con el que se construye arquitectura (source: estilos-arquitectonicos.md).

FUENTE

Referencia clásica: *Buschmann, Meunier, Rohnert, Sommerlad, Stal — Pattern-Oriented Software Architecture (POSA) Vol. 1-5* (source: estilos-arquitectonicos.md).

§1 Familia Dataflow

Los componentes se organizan alrededor de *transformaciones sucesivas de datos*.

DEFINICIÓN

Dataflow: el dato fluye etapa a etapa; cada componente lee, transforma y pasa el resultado al siguiente. Incluye *Batch Sequential*, *Pipes & Filters* y *Hierarchical Layers* (source: estilos-arquitectonicos.md).

Pipes & Filters

Filtros (componentes) conectados por pipes (streams). Cada filtro lee, transforma, escribe. Composable y paralelizable (source: estilos-arquitectonicos.md).

CUÁNDO SÍ · VENTAJAS

Throughput batch alto, etapas reutilizables y combinables, paralelización natural por filtro.

CUÁNDO NO · CUIDADO

Latencia interactiva baja: el dato debe cruzar todos los filtros. Mal ajuste para request/response sincrónico.

EJEMPLO

Unix pipes, Apache NiFi. Batch Sequential (variante): ETL nocturnos, procesamiento mainframe — cada etapa procesa el dataset completo antes de pasarlo (source: estilos-arquitectonicos.md).

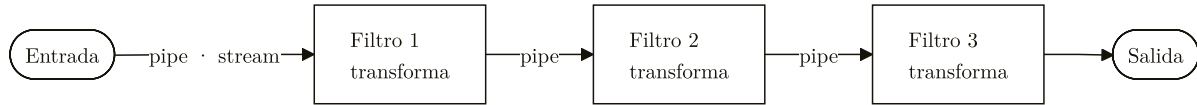


FIGURA 1. *Pipes & Filters*: el dato fluye filtro a filtro por pipes (streams); cada filtro lee, transforma y escribe.

Hierarchical Layers

Capas ordenadas donde cada capa usa sólo la inmediata inferior (source: estilos-arquitectonicos.md).

CUÁNDO SÍ • VENTAJAS

Maintainability y separación de responsabilidades; performance buena con in-process calls. Base de monolitos y de Hexagonal / Clean / Onion.

CUÁNDO NO • CUIDADO

Capas estrictas **fuerzan una forma que no encaja** en sistemas event-driven (anti-patrón, source: estilos-arquitectonicos.md).

EJEMPLO

OSI networking; capas clásicas de app: presentación → lógica → datos (source: estilos-arquitectonicos.md).

§2 Familia Distributed

Los componentes están *separados por red* y coordinan su interacción.

DEFINICIÓN

Distributed: componentes distribuidos que se coordinan vía intermediarios o suscripción. Incluye *Broker*, *Publish-Subscribe*, *Forwarder-Receiver* y *Client-Dispatcher-Server* (source: estilos-arquitectonicos.md).

Broker

Un componente intermediario coordina la comunicación entre clientes y servidores (source: estilos-arquitectonicos.md).

CUÁNDO SÍ • VENTAJAS

Desacopla cliente de la ubicación/identidad del servidor; centraliza ruteo y mediación.
Habilita scalability horizontal.

EJEMPLO

CORBA; message brokers como RabbitMQ y Kafka (source: estilos-arquitectonicos.md).

Publish-Subscribe

Emisores **publican eventos**; receptores se **suscriben por tópico**. Desacopla temporalmente emisor y receptor (source: estilos-arquitectonicos.md).

CUÁNDO SÍ • VENTAJAS

SCALABILITY

AVAILABILITY

Scalability horizontal y, con replicación, fault tolerance.

Productor y consumidor no necesitan estar vivos al mismo tiempo.

CUÁNDO NO • CUIDADO

Trazar el flujo extremo a extremo se vuelve difícil (acoplamiento implícito por tópicos);
orden y entrega exactly-once requieren cuidado.

EJEMPLO

Kafka, Redis Pub/Sub, MQTT (source: estilos-arquitectonicos.md). Columna vertebral de Event-Driven Architecture (EDA).

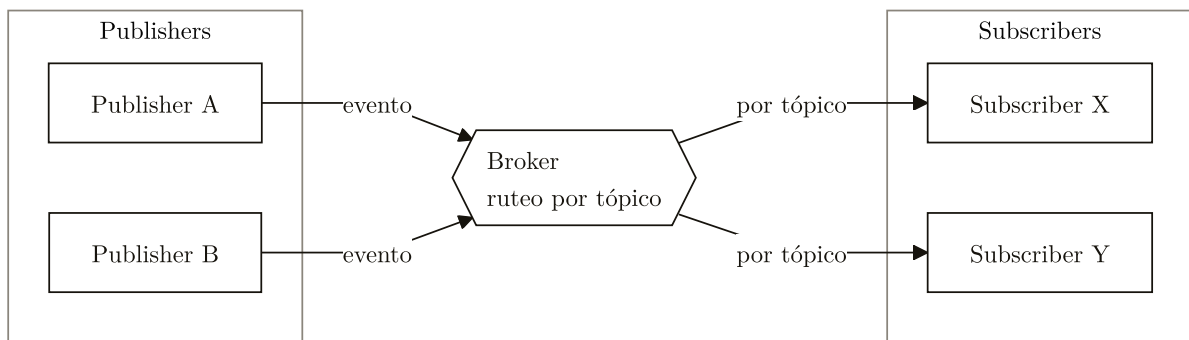


FIGURA 2. Publish-Subscribe: emisores publican eventos a un broker que los rutea por tópico a los suscriptores, desacoplando emisor y receptor.

Forwarder-Receiver: intermediarios que encapsulan los detalles de la comunicación de red. **Client-Dispatcher-Server:** el dispatcher resuelve qué server atiende a qué cliente — precursor del service discovery moderno (source: estilos-arquitectonicos.md).

§3 Familia Interactive

Separan *presentación, lógica de control y datos*.

DEFINICIÓN

Interactive (UI): familia que aísla la vista de los datos y de la lógica de control.

Variantes principales: **MVC**, **MVP**, **MVVM** (también PAC y VIPER) (source: estilos-arquitectonicos.md).

MVC — Model View Controller

Clásico. Modelo (datos), Vista (UI), Controlador (lógica de entrada) (source: estilos-arquitectonicos.md).

MVP — Model View Presenter

Variante de MVC: el Presenter maneja toda la lógica y la Vista es **pasiva**. Facilita testing (source: estilos-arquitectonicos.md).

MVVM — Model View ViewModel

El ViewModel expone estado **databindeable** a la Vista. Popular en WPF, Angular, Vue (source: estilos-arquitectonicos.md).

CUÁNDO SÍ • VENTAJAS

Maintainability y testability: separar la vista de la lógica permite testear el comportamiento sin la UI (especialmente MVP/MVVM).

CUÁNDO NO • CUIDADO

MVC "de libro de texto" como default: las variantes modernas (MVP/MVVM) suelen ser mejores para testing (anti-patrón, source: estilos-arquitectonicos.md).



FIGURA 3. MVC vs MVVM: en MVC el Controller media entre View y Model; en MVVM la View pasiva se databindea a un ViewModel observable.

Otras: **PAC** (composición jerárquica de agentes Presentation-Abstraction-Control, para UIs complejas con sub-ventanas independientes); **VIPER** (View/Interactor/Presenter/Entity/Router, modular en iOS) (source: estilos-arquitectonicos.md).

§4 Familia Event-based

Componentes que reaccionan a **eventos asincrónicos**.

DEFINICIÓN

Event-based: la unidad de coordinación es el **evento**. Cuatro modos por complejidad de procesamiento: **SEP**, **STREP**, **CEP**, **OLEP** (source: estilos-arquitectonicos.md).

TABLA 1. Los cuatro modos de procesamiento de eventos

Modo	Qué hace	Ejemplo
SEP — Simple Event Processing	Cada evento se procesa individualmente	Listeners clásicos en GUI
STREP — Stream Event Processing	Streams continuos: ventanas temporales, agregados	Apache Flink, Kafka Streams
CEP — Complex Event Processing	Detecta patrones entre múltiples eventos	"3 intentos fallidos en 60s desde misma IP"
OLEP — Online Event Processing	Event log como fuente de verdad; servicios derivan estado	Kleppmann (2019); base de Event Sourcing

CUÁNDO SÍ • VENTAJAS

SCALABILITY Evolvability y desacoplamiento; real-time analytics (STREP/CEP); audit y reproducibilidad (OLEP / Event Sourcing) (source: estilos-arquitectonicos.md).

CUÁNDO NO • CUIDADO

Razonar sobre el estado global y depurar flujos asincrónicos es difícil; consistencia eventual y reproceso del log agregan complejidad.

EJEMPLO

CEP en seguridad: detectar el patrón "3 logins fallidos en 60s desde la misma IP" requiere correlacionar varios eventos, no uno (source: estilos-arquitectonicos.md).

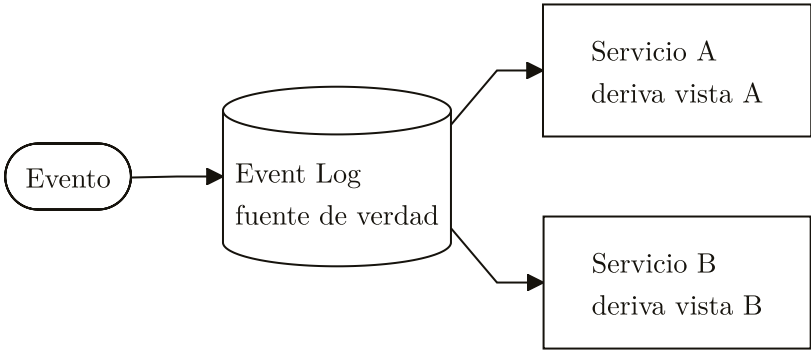


FIGURA 4. OLEP: el event log es la fuente de verdad y cada servicio deriva su propia vista a partir del mismo log.

§5 Otras familias y estilos compuestos

Otras familias mencionadas

- **Peer-to-Peer** — nodos equivalentes intercambian trabajo/datos sin servidor central.
- **Blackboard** — espacio de conocimiento compartido; múltiples solvers cooperan. IA clásica.
- **Microkernel / Plug-in** — core mínimo + extensiones. Eclipse, VSCode, kernels OS.
- **Reflection** — el sistema razona sobre sí mismo y se modifica en runtime.

Compuestos (macro-arquitecturas)

- **Monolítico** — todo en un proceso; suele usar Layers internamente.
- **Microservicios** — servicios con Broker/Pub-Sub/REST, cada uno con layers internos.
- **Serverless (FaaS)** — funciones reactivas a eventos.
- **EDA** — pub-sub + CEP como columna vertebral.
- **CQRS + Event Sourcing** — separa lectura/escritura con OLEP de backing store.
- **Hexagonal / Clean / Onion** — Layers con énfasis en independencia del dominio.

CUÁNDO NO • CUIDADO

"Compuesto" no es magia: a veces se llaman "estilos" pero son **combinaciones** de los anteriores aplicadas al sistema completo. Cada estilo agrega complejidad; combinar 4 "porque son modernos" es caos (source: estilos-arquitectonicos.md).

§6 Cómo elegir — tabla resumen

Para cada decisión de estilo, aplicar **Attribute Driven Design (ADD)**: (1) identificar atributos priorizados, (2) evaluar qué soporta/compromete cada candidato, (3) hacer explícitos los trade-offs, (4) elegir el que cubre los drivers dominantes ([Attribute Driven Design], source: estilos-arquitectonicos.md).

TABLA 2. Atributo dominante → estilo favorecido

Atributo dominante	Estilo típicamente favorecido
Performance (latencia baja)	Hierarchical Layers, in-process calls
Scalability horizontal	Pub-Sub, Broker, Microservicios
Availability / Fault tolerance	Pub-Sub con replicación, Peer-to-Peer
Maintainability	Layers, MVC/MVP/MVVM, Microkernel
Evolvability	Event-driven + CQRS, Microkernel
Throughput batch	Batch Sequential, Pipes & Filters
Real-time analytics	STREP, CEP
Audit y reproducibilidad	OLEP / Event Sourcing

Tabla literal de la fuente (source: estilos-arquitectonicos.md).

CUÁNDO NO · CUIDADO

Anti-patrones (source: estilos-arquitectonicos.md): "Microservicios por default" (complejidad operacional desproporcionada); Layers estrictos sobre sistemas event-driven; MVC "de libro" cuando MVP/MVVM testean mejor; mezclar estilos sin justificación.

REGLA DE ORO

Ningún estilo es *silver bullet* (Brooks). Cada uno trae trade-offs conocidos: la pregunta no es "cuál es el mejor" sino "cuál cubre los **drivers dominantes** de este sistema".

EN EL PARCIAL

Si piden elegir un estilo: nombrar los atributos priorizados primero, mapearlos a 1-2 estilos candidatos vía la tabla, hacer explícito al menos un trade-off y justificar la elección por los drivers. Saltar al estilo sin nombrar atributos mata la respuesta.